Lab Activity Documentation: Scalable Vector Graphics (SVG) Implementation

1. Objectives

- **Integrate Vector Graphics:** Learn how to embed scalable vector graphics directly into web pages using the HTML `<svg>` element.
-
- **Configure Canvas Dimensions:** Set explicit width and height properties to define the display size of the graphic vector element.
-
- **Manage Viewport Scaling:** Utilize the `viewBox` attribute to define the internal coordinate system and ensure the vector graphic scales smoothly across different screen sizes without losing quality.
-
- **Render Vector Paths:** Use the `<path>` element combined with complex SVG path data (`d` attribute) to draw custom shapes like a heart icon.
-

2. Lab Implementation Overview

During this lab, I built a lightweight web document demonstrating how to render inline vector icons:

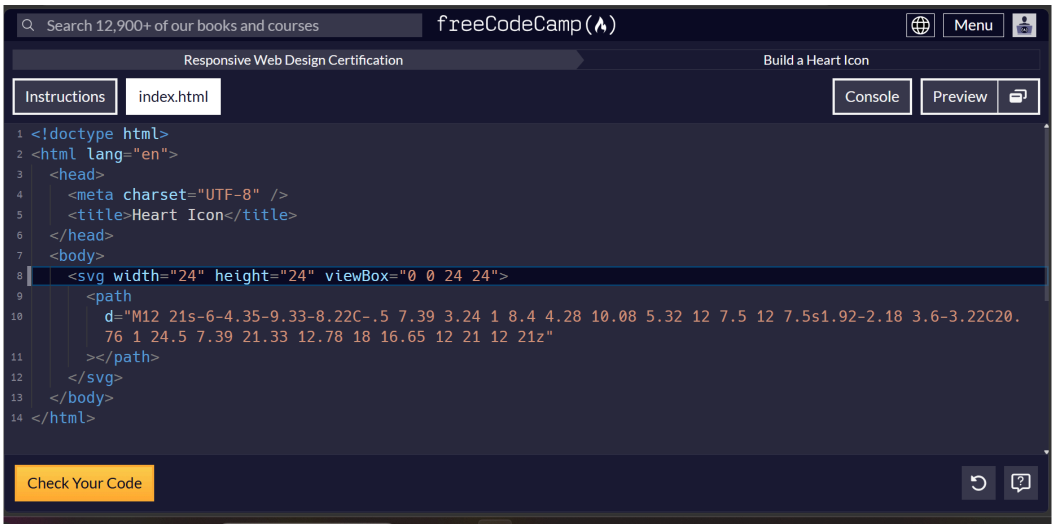
- **Document Setup:** Established a clean HTML5 template with proper metadata, character encoding, and a descriptive title ("Heart Icon").
-
- **SVG Canvas Configuration:** Inserted an `<svg>` element with dimensions set to `width="24"` and `height="24"` pixels.
-
- **ViewBox Mapping:** Configured the `viewBox="0 0 24 24"` attribute to establish a 24x24 coordinate space, ensuring the graphic scales responsively while maintaining its precise proportions.
-
- **Path Rendering:** Embedded a `<path>` element utilizing specific coordinate commands in the `d` attribute to accurately plot and render a heart shape directly in the browser.
-

3. Conclusion & Key Takeaways

Completing this lab significantly enhanced my understanding of vector-based graphics in web development:

- **Resolution Independence:** I learned that unlike raster images (like PNG or JPG), SVGs scale infinitely without pixelation, making them ideal for UI icons.
-
- **Inline Control:** I gained practical experience embedding graphics directly into the HTML markup, which allows for easy styling and color manipulation using CSS.
-
- **Path Data Understanding:** I now have a clearer appreciation of how coordinate-based path strings (`d` attributes) are utilized by browsers to render complex vector shapes.

CODE:



PREVIEW: